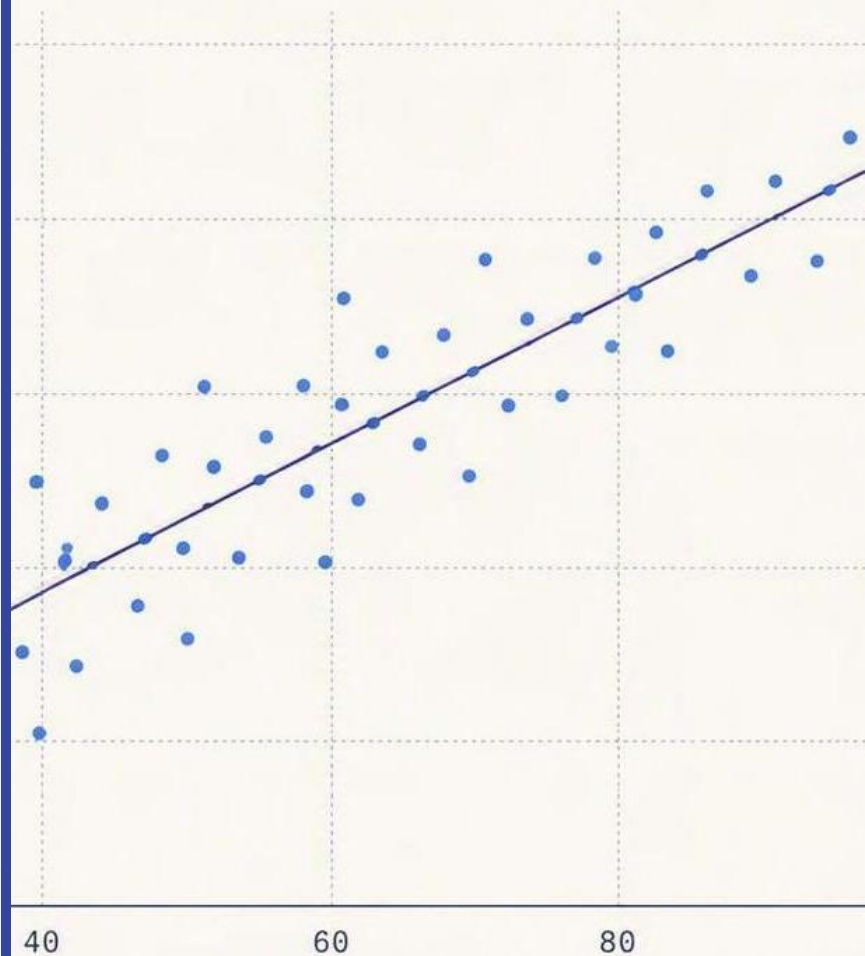


MACHINE LEARNING • MODELOS PREDICTIVOS

Algoritmos de Regresión en Machine Learning

Cómo funcionan, cuándo utilizarlos y cómo compararlos.

Si tengo que predecir un valor numérico, ¿qué algoritmo debería elegir?



¿Qué es un problema de regresión?

La regresión predice una variable objetivo numérica y continua a partir de un conjunto de características de entrada.

A diferencia de la clasificación, la salida no es una categoría: es un número dentro de un rango, y buscamos que la predicción se acerque lo máximo posible al valor real.

$$y = f(x_1, x_2, \dots, x_n) + \text{error}$$

Precio de una casa

Ventas futuras

Temperatura

Tiempo de espera

Consumo energético

Valor de una acción

Variable objetivo numérica → un número, no una etiqueta.

Clasificación vs. Regresión

CLASIFICACIÓN

Predice categorías discretas

- ▶ ¿La casa se venderá este mes? Sí / No
- ▶ Segmento de precio: bajo / medio / alto
- ▶ Salida = clase o etiqueta
- ▶ Se evalúa con accuracy, F1, AUC

REGRESIÓN

Predice un valor numérico continuo

- ▶ ¿En cuánto se venderá la casa? \$312.400
- ▶ Cualquier valor dentro de un rango
- ▶ Salida = número real
- ▶ Se evalúa con MAE, RMSE, R^2

Misma casa, dos preguntas distintas: la clasificación separa; la regresión estima magnitud.

Los algoritmos que veremos

Lineales y regularizados

- Regresión Lineal
- Polinómica
- Ridge
- Lasso
- Elastic Net

Basados en árboles

- Árbol de Regresión
- Random Forest
- Gradient Boosting
- XGBoost / LightGBM

Basados en instancias / margen / redes

- SVR
- KNN Regressor
- Redes Neuronales (MLP)

12 familias · un mismo objetivo: predecir un número continuo.

¿Cómo elegir un modelo de regresión?

Cantidad de datos

Pocos → modelos simples; muchos → boosting / redes

Complejidad de la relación

Lineal vs. no lineal e interacciones

Interpretabilidad

¿Necesito explicar cada predicción?

Velocidad

Entrenamiento e inferencia en producción

Precisión

Error objetivo aceptable para el negocio

Overfitting

Riesgo de memorizar el ruido

Escalabilidad

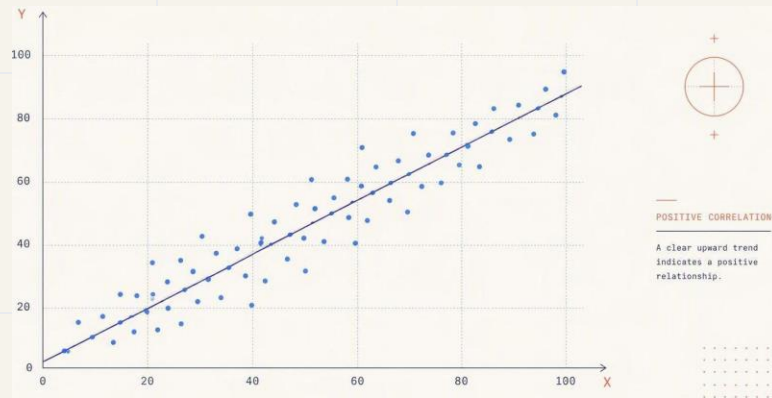
Volumen, dimensionalidad y coste de cómputo

No existe un ganador universal: cada factor empuja hacia una familia distinta. La comparativa final resume todos estos ejes.

Regresión Lineal: la mejor recta

Trazar la mejor recta posible que resuma la relación entre las variables y el precio.

- Cada punto es una casa; la recta captura la tendencia media.
- “Mejor” = la que minimiza el error cuadrático (mínimos cuadrados).
- El modelo base contra el que se comparan los demás.



Una ecuación lineal simple

$$\hat{y} = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_n \cdot x_n$$

b_0 = intercepto

b_i = pendientes / pesos

Cada coeficiente b_i dice cuánto cambia el precio si la variable i sube una unidad, manteniendo el resto constante.

El signo indica la dirección; la magnitud, la importancia. Por eso el modelo es tan fácil de leer.

Lectura de negocio

- +150 €/m² por cada metro extra
- +12.000 € por habitación adicional
- Coeficientes = reglas transparentes

Cuándo usar Regresión Lineal

Ideal como primer modelo y siempre que la interpretabilidad importe tanto como la precisión.

Relación ~lineal

El objetivo crece de forma aproximadamente proporcional.

Interpretabilidad

Necesitas explicar el porqué de cada predicción.

Línea base

Punto de partida antes de probar modelos complejos.

Negocio y finanzas

Pricing, elasticidad, riesgo, forecasting simple.

Pocas variables

Con más filas que columnas y baja multicolinealidad.

Coste bajo

Entrena en milisegundos y es fácil de mantener.

Señal para cambiar de modelo: coeficientes inestables o residuos con patrón claro.

Regresión Lineal: balance

VENTAJAS

- ✓ Muy simple y rápida de entrenar
- ✓ Altamente interpretable (coeficientes)
- ✓ Sin apenas hiperparámetros
- ✓ Excelente línea base de referencia

LIMITACIONES

- ✗ No capta relaciones no lineales
- ✗ Sensible a outliers y multicolinealidad
- ✗ Asume relación aditiva entre variables
- ✗ Requiere ingeniería manual de features

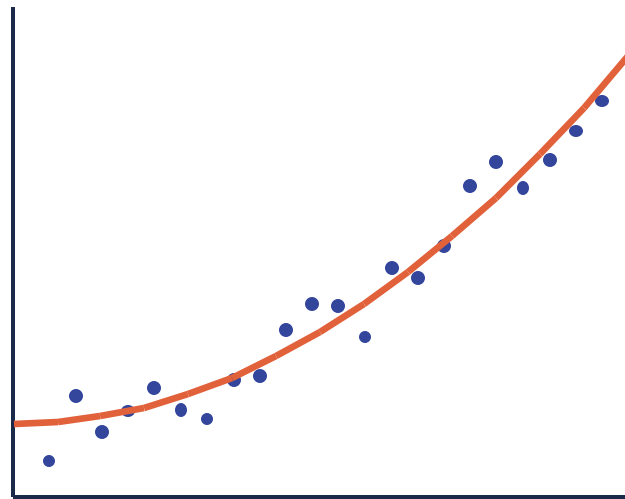
EJEMPLO REAL • PRECIO DE VIVIENDAS

En Ames Housing, una regresión regularizada (Ridge) alcanza $\sim 0,137$ de RMSLE: una base sólida que los modelos de árboles luego mejoran.

Regresión Polinómica: una curva

Cuando una recta se queda corta, ajustamos una curva suave a los datos.

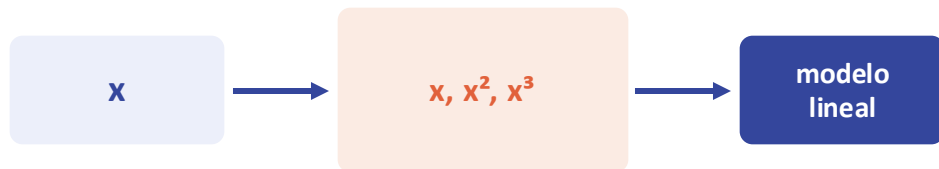
- Sigue siendo un modelo lineal... en variables transformadas.
- Captura relaciones que se aceleran o se saturan.
- El grado del polinomio controla cuánto se curva.



Curva de grado 2

Transformación de variables

$$\hat{y} = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots$$



Generamos términos al cuadrado, al cubo, etc. y aplicamos una regresión lineal sobre esas nuevas columnas. La relación se vuelve curva sin cambiar el algoritmo.

Claves

- Grado 2–3 suele bastar
- Escalar features antes de elevar
- Grados altos → oscilaciones

Cuándo usar Regresión Polinómica

Para relaciones curvilíneas simples y conocidas, sin renunciar del todo a la interpretabilidad.

Curvatura conocida

Sabes que la relación se acelera o se frena.

Pocas variables

Funciona mejor con 1–3 predictores clave.

Efectos de saturación

Rendimientos marginales decrecientes.

Física / ingeniería

Leyes cuadráticas o cúbicas conocidas.

Mejorar la lineal

Cuando la recta deja patrón en los residuos.

Datos limpios

Sensible a outliers en los extremos.

Regresión Polinómica: balance

VENTAJAS

- ✓ Captura relaciones no lineales suaves
- ✓ Sigue siendo relativamente interpretable
- ✓ Reutiliza toda la maquinaria lineal
- ✓ Rápida con pocos predictores

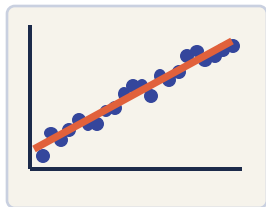
LIMITACIONES

- ✗ Grados altos → sobreajuste severo
- ✗ Extrapola muy mal fuera del rango
- ✗ Explota con muchas variables
- ✗ Muy sensible a la escala y outliers

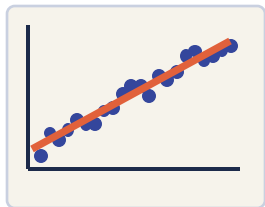
Regla práctica: sube el grado sólo si la validación cruzada lo justifica.

Sobreajuste y regularización

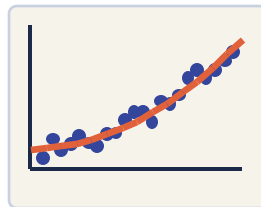
Un modelo demasiado flexible memoriza el ruido: acierta en entrenamiento y falla en datos nuevos.



Subajuste



Ade cuado



Sobreajuste

La regularización penaliza coeficientes grandes para mantener el modelo simple y estable.

Regularización

- Añade una penalización al error
- Frena coeficientes desbocados
- Reduce varianza, mejora generalización
- λ controla la intensidad

Ridge: regularización L2

$$\min \quad \Sigma(y - \hat{y})^2 + \lambda \cdot \Sigma b_i^2$$

error de ajuste

penalización L2 (b_i^2)

Ridge encoge todos los coeficientes hacia cero de forma proporcional, pero nunca los anula del todo.

λ alto $\rightarrow$ más encogimiento y más sesgo; $\lambda = 0 \rightarrow$ regresión lineal ordinaria.

En una frase

- Reparte peso entre variables correlacionadas
- Mantiene todas las features
- Estabiliza los coeficientes

Cuándo usar Ridge

Cuando hay muchas variables, algunas correlacionadas entre sí, y todas parecen aportar algo.

Multicolinealidad

Variables muy correlacionadas (m^2 , superficie total).

Muchas features

Más predictores, riesgo de sobreajuste.

Todo relevante

No quieres eliminar variables, sólo moderarlas.

Coeficientes inestables

La lineal da pesos enormes de signo cambiante.

Datos ruidosos

Necesitas un modelo más estable.

Interpretación suave

Sigues leyendo coeficientes, ya domados.

Ridge: balance

VENTAJAS

- ✓ Controla el sobreajuste eficazmente
- ✓ Estable con features correlacionadas
- ✓ Mantiene toda la información
- ✓ Solución cerrada y rápida

LIMITACIONES

- ✗ No hace selección de variables
- ✗ Requiere ajustar λ por validación
- ✗ Necesita escalar las variables
- ✗ Menos interpretable que Lasso

Ridge no elimina variables: si buscas un modelo disperso, mira Lasso.

Lasso: regularización L1

$$\min \sum (y - \hat{y})^2 + \lambda \cdot \sum |b_i|$$

error de ajuste

penalización L1 ($|b_i|$)

El valor absoluto empuja algunos coeficientes exactamente a cero.

Resultado: un modelo disperso que se queda sólo con las variables que de verdad importan.

L1 vs. L2

- Ridge encoge; Lasso elimina
- L1 = selección automática
- Ideal si sospechas features inútiles

Lasso elimina lo irrelevante

Superficie (m²)



Nº habitaciones



Barrio



Color de la puerta

→ 0 (eliminada)

Nº de la casa

→ 0 (eliminada)

Antigüedad



Qué observamos

- Variables sin señal → coeficiente 0
- El modelo se queda con lo esencial
- Más fácil de explicar y desplegar
- Menos coste de recogida de datos

Cuándo usar Lasso

Cuando tienes muchas variables candidatas y quieres un modelo disperso, interpretable y automático.

Alta dimensionalidad

Cientos de features, pocas realmente útiles.

Selección automática

Que el modelo elija las variables por ti.

Modelos interpretables

Pocas variables con peso claro.

Reducir coste de datos

Menos variables que medir en producción.

Genómica / texto / IoT

Miles de columnas, señal dispersa.

Prototipado rápido

Descubrir qué variables importan.

Lasso: balance

VENTAJAS

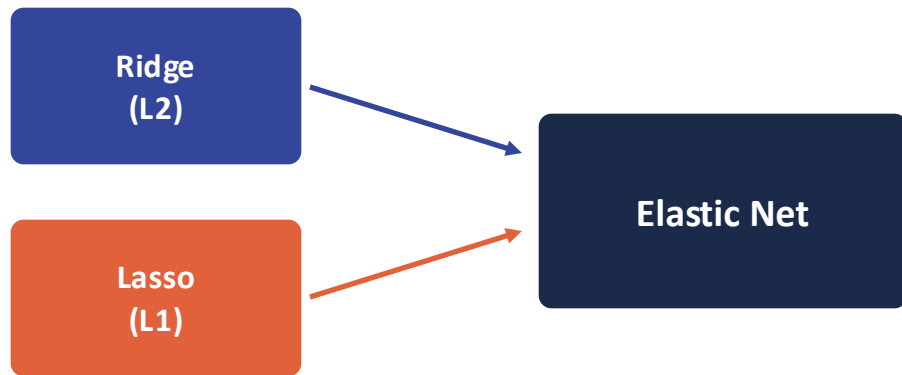
- ✓ Selección automática de variables
- ✓ Modelos dispersos e interpretables
- ✓ Controla el sobreajuste
- ✓ Ideal en alta dimensionalidad

LIMITACIONES

- ✗ Con grupos correlacionados elige una al azar
- ✗ Puede descartar variables útiles
- ✗ Inestable si features muy correlacionadas
- ✗ Requiere escalar y ajustar λ

¿Grupos de variables correlacionadas? Elastic Net combina lo mejor de Ridge y Lasso.

Elastic Net: Ridge + Lasso



Combina ambas penalizaciones: encoge coeficientes (L2) y a la vez elimina los irrelevantes (L1).

Lo mejor de dos mundos

- Selección (como Lasso)
- Estabilidad (como Ridge)
- Maneja grupos correlacionados

Dos penalizaciones, dos mandos

$$\min \sum (y - \hat{y})^2 + \lambda [\alpha \cdot \sum |b_i| + (1-\alpha) \cdot \sum b_i^2]$$

$\lambda \rightarrow$ intensidad total de la regularización.

$\alpha \rightarrow$ mezcla entre L1 y L2:

$\alpha = 1 \rightarrow$ Lasso puro

$\alpha = 0 \rightarrow$ Ridge puro

Ajustando α y λ por validación cruzada encuentras el punto óptimo entre selección y estabilidad.

Cómo se ajusta

- Grid o búsqueda sobre (α, λ)
- Validación cruzada k-fold
- Escalar siempre las variables

Cuándo usar Elastic Net

El regularizado por defecto cuando hay muchas variables correlacionadas en grupos.

Grupos correlacionados

Selecciona grupos, no una variable arbitraria.

$p \gg n$

Más variables que observaciones.

Selección + estabilidad

Quieres dispersión sin la inestabilidad de Lasso.

Datos económicos

Indicadores redundantes y correlacionados.

Bioinformática

Genes co-expresados en bloques.

Duda Ridge/Lasso

Elastic Net como punto intermedio seguro.

Elastic Net: balance

VENTAJAS

- ✓ Combina selección y estabilidad
- ✓ Maneja grupos de variables correlacionadas
- ✓ Robusto en alta dimensionalidad
- ✓ Generaliza Ridge y Lasso

LIMITACIONES

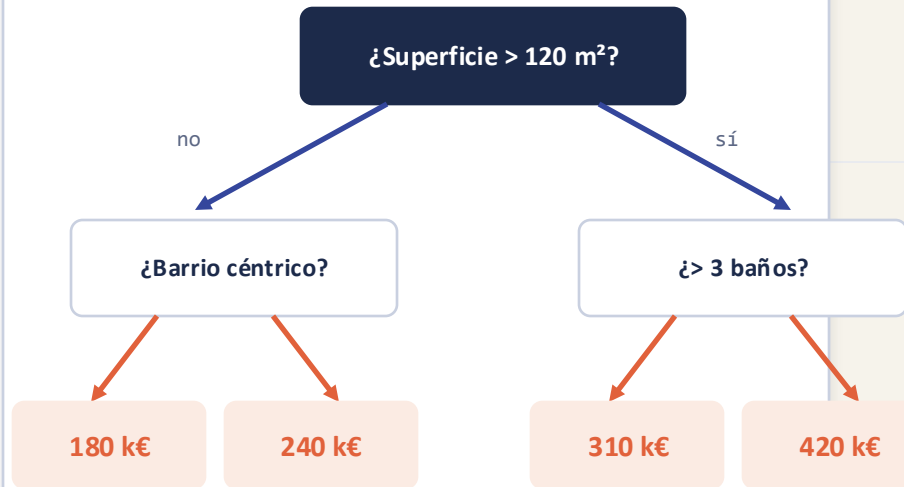
- ✗ Dos hiperparámetros que ajustar (α , λ)
- ✗ Sigue siendo un modelo lineal
- ✗ Menos intuitivo de explicar
- ✗ Requiere escalado y validación cuidadosa

Cierra la familia lineal: a partir de aquí, entramos en modelos no lineales.

Árbol de Regresión: preguntas encadenadas

Un árbol de decisiones que divide las casas con preguntas sí/no y predice un precio por grupo.

- Imita cómo razona un tasador: filtra por reglas.
- Cada hoja = un grupo de casas con precio medio.
- Capta relaciones no lineales sin ecuaciones.



Particiones sucesivas del espacio

- 1 Busca la variable y el punto de corte que más reducen el error.
- 2 Divide los datos en dos ramas por ese umbral.
- 3 Repite el proceso en cada rama de forma recursiva.
- 4 Se detiene por profundidad, mínimo de muestras o ganancia.
- 5 Cada hoja predice el promedio de su grupo de casas.

Sin preparación previa

- No necesita escalar variables
- Maneja no linealidades e interacciones
- Acepta variables categóricas
- Fácil de visualizar y explicar

Cuándo usar un Árbol de Regresión

Cuando la relación es claramente no lineal y quieres un modelo visual y explicable.

Relaciones no lineales

Umbrales y saltos en el precio.

Interacciones

El efecto de una variable depende de otra.

Explicabilidad visual

Mostrar el árbol a un equipo de negocio.

Datos mixtos

Numéricos y categóricos sin preprocesar.

Reglas de decisión

Segmentar clientes o propiedades.

Prototipo rápido

Base para Random Forest o Boosting.

Árbol de Regresión: balance

VENTAJAS

- ✓ Muy interpretable y visual
- ✓ Capta no linealidades e interacciones
- ✓ Sin escalado ni supuestos
- ✓ Acepta datos mixtos

LIMITACIONES

- ✗ Fuerte tendencia al sobreajuste
- ✗ Inestable: pequeños cambios, otro árbol
- ✗ Predicciones escalonadas, no suaves
- ✗ Un solo árbol pierde precisión

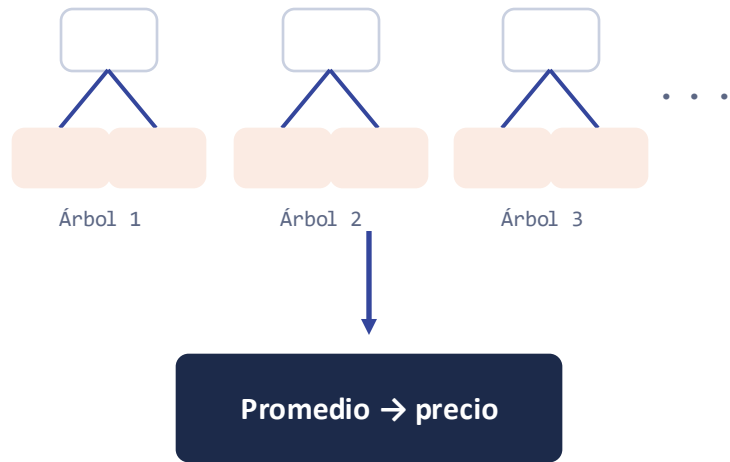
EJEMPLO REAL • PRECIO DE VIVIENDAS

El sobreajuste de un árbol suelto se resuelve combinando muchos: así nacen Random Forest y Gradient Boosting.

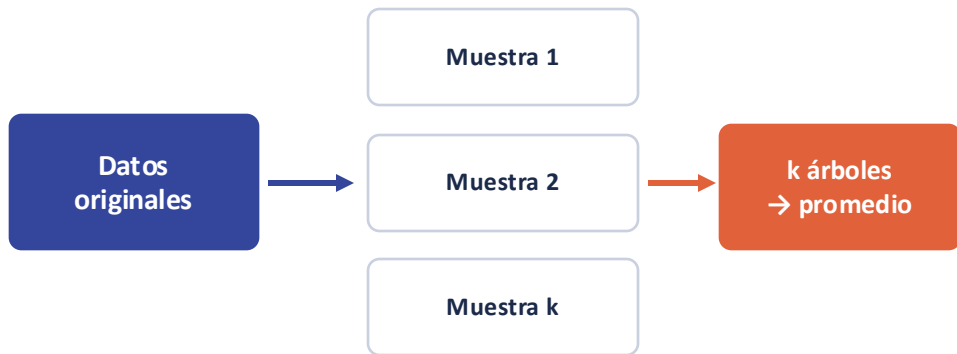
Random Forest: la sabiduría del bosque

Muchos árboles distintos votan y su promedio da una predicción más fiable que cualquiera solo.

- Cada árbol ve datos y variables ligeramente distintos.
- Los errores individuales se cancelan al promediar.
- “Sabiduría de la multitud” aplicada a modelos.



Bagging: bootstrap + promedio



Bootstrap: cada árbol se entrena con una muestra aleatoria con reemplazo y un subconjunto de variables. Al promediar, baja la varianza sin subir el sesgo.

Tres ingredientes

- Bootstrap: muestreo con reemplazo
- Aleatoriedad de features por nodo
- Agregación: promedio de árboles
- Error out-of-bag para validar

Cuándo usar Random Forest

Un excelente algoritmo general: sólido, robusto y con poca puesta a punto.

Modelo de referencia fuerte

Gran rendimiento con parámetros por defecto.

No linealidad + interacciones

Sin especificarlas a mano.

Datos ruidosos

Robusto frente a outliers y ruido.

Importancia de variables

Ranking de features útil para negocio.

Poco tuning disponible

Funciona bien casi sin ajustar.

Datos tabulares medianos

Miles a cientos de miles de filas.

Suele superar a un árbol único y compite de cerca con el boosting con menos esfuerzo.

Random Forest: balance

VENTAJAS

- ✓ Muy robusto y preciso sin apenas tuning
- ✓ Reduce el sobreajuste del árbol único
- ✓ Da importancia de variables
- ✓ Paraleliza fácilmente

LIMITACIONES

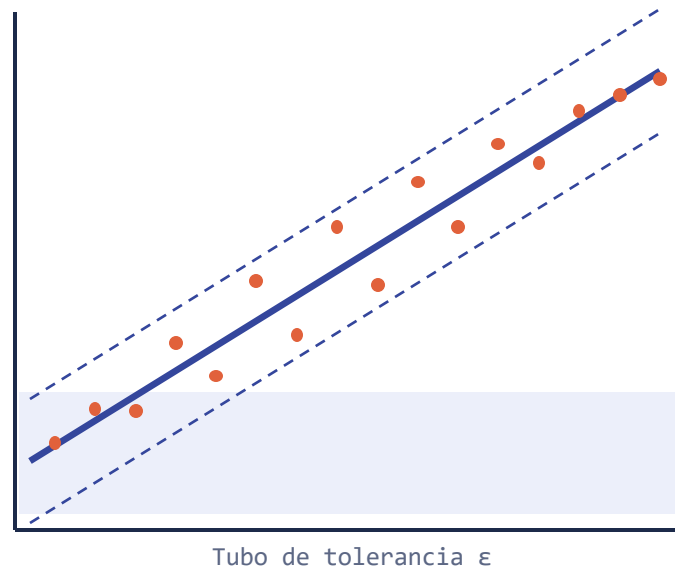
- ✗ Menos interpretable (caja gris)
- ✗ Modelos pesados en memoria
- ✗ Inferencia más lenta que un árbol
- ✗ Suele quedar por debajo del boosting

Bagging entrena árboles en paralelo; el boosting los entrena en secuencia para afinar el error.

SVR: Support Vector Regression

En vez de minimizar todos los errores, SVR busca una función que quepa dentro de un tubo de tolerancia.

- Ignora los errores pequeños (dentro del margen ϵ).
- Sólo penaliza los puntos que se salen del tubo.
- Con kernels, modela relaciones muy no lineales.



Margen y error tolerado

Tubo ϵ : banda alrededor de la predicción donde el error se considera cero.

Vectores de soporte: sólo los puntos fuera del tubo definen la función. El resto no influye.

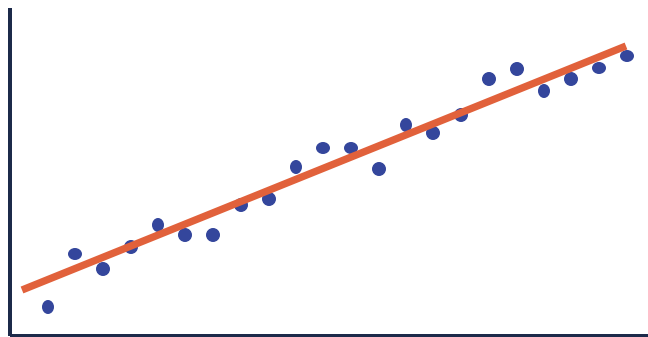
Parámetro C: controla cuánto penalizamos las desviaciones grandes (sesgo-varianza).

Intuición

- “Acierta dentro de un margen, no exacto”
- Robusto a pequeños errores y ruido
- ϵ y C se ajustan por validación

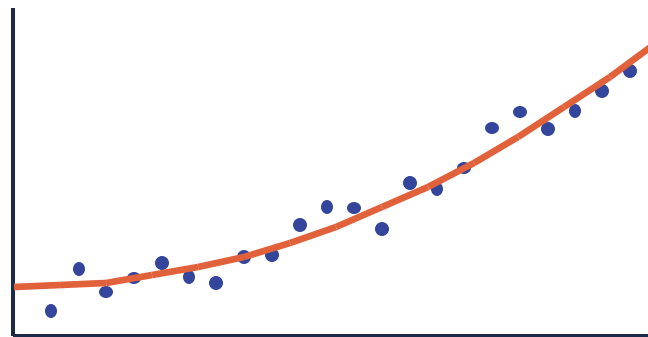
Kernel lineal y kernel RBF

Kernel lineal



Relaciones aproximadamente lineales; rápido y simple.

Kernel RBF



Relaciones curvas y complejas; más potente, más tuning (γ).

Cuándo usar SVR

Brilla en datasets pequeños o medianos con relaciones no lineales y features bien escaladas.

Datos pequeños/medianos

Hasta decenas de miles de filas.

No linealidad

Con kernel RBF captura curvas complejas.

Robustez a outliers

El margen ϵ amortigua errores pequeños.

Alta dimensión moderada

Funciona bien con muchas features.

Precisión sobre velocidad

Cuando puedes permitir más cómputo.

Features escaladas

Requiere normalización previa.

SVR: balance

VENTAJAS

- ✓ Eficaz en no linealidad con kernels
- ✓ Robusto frente a outliers (margen ϵ)
- ✓ Bueno en alta dimensión moderada
- ✓ Sólido en datasets pequeños

LIMITACIONES

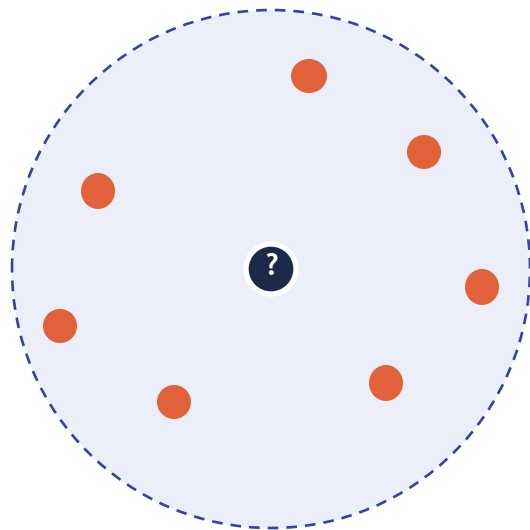
- ✗ No escala a grandes volúmenes
- ✗ Sensible a ϵ , C y γ (mucho tuning)
- ✗ Exige escalar las variables
- ✗ Poco interpretable

Con millones de filas, el boosting o LightGBM suelen ser mejores opciones que SVR.

KNN: mirar a los vecinos parecidos

Para tasar una casa, mira las k casas más parecidas y promedia sus precios.

- “Casas similares valen precios similares.”
- No entrena un modelo: memoriza los datos.
- k controla cuántos vecinos se consultan.



k vecinos más cercanos

Promedio de los vecinos próximos

- 1 Mide la distancia entre la casa nueva y todas las demás.
- 2 Selecciona las k casas más cercanas (vecinos).
- 3 Predice el promedio de sus precios.
- 4 Opcional: pondera por cercanía (vecinos más cerca pesan más).

Detalles clave

- k pequeño $\rightarrow$ ruidoso; k grande $\rightarrow$ suaviza
- Escalado obligatorio (distancias)
- Coste alto en inferencia
- Sufre la maldición de la dimensión

Cuándo usar KNN Regressor

Útil cuando la señal es local y la geometría de los datos importa más que una fórmula global.

Patrones locales

Vecindarios homogéneos, efectos de proximidad.

Pocas dimensiones

Rinde mejor con pocas variables.

Datos densos

Muchos ejemplos por región del espacio.

Sin supuestos

No asume forma de la relación.

Baseline no lineal

Rápido de probar como referencia.

Recomendación / geoprecio

Casos con similitud natural.

KNN Regressor: balance

VENTAJAS

- ✓ Muy simple e intuitivo
- ✓ Sin fase de entrenamiento
- ✓ Captura estructura local no lineal
- ✓ Sin supuestos sobre los datos

LIMITACIONES

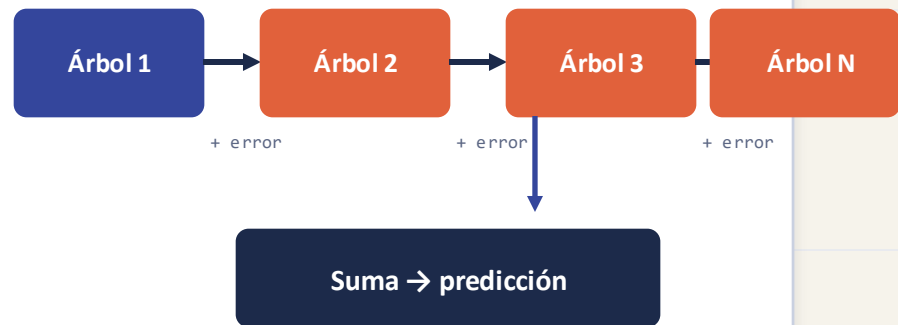
- ✗ Inferencia lenta con muchos datos
- ✗ Sensible a la escala y a k
- ✗ Sufre la maldición de la dimensionalidad
- ✗ Malo extrapolando fuera de lo visto

Guarda todo el dataset: la memoria y la latencia crecen con cada nuevo ejemplo.

Gradient Boosting: corregir errores

Cada nuevo árbol se especializa en corregir los errores que dejaron los anteriores.

- Editor iterativo: mejora la predicción paso a paso.
- Empieza simple y refina donde más falla.
- La tasa de aprendizaje modera cada corrección.



Ensamblado secuencial

1

Predice un valor inicial simple (p. ej. el precio medio).

2

Calcula el error residual de cada casa.

3

Entrena un árbol pequeño para predecir ese residuo.

4

Suma su corrección (escalada por la tasa η).

5

Repite cientos de veces: el error baja gradualmente.

Boosting vs. Bagging

- Bagging: árboles en paralelo, se promedian
- Boosting: en secuencia, corrigen el error
- η (learning rate): 0,01–0,3
- Más árboles $\rightarrow$ más ajuste (y riesgo)

Cuándo usar Gradient Boosting

La opción por defecto cuando buscas la máxima precisión en datos tabulares.

Máxima precisión

Suele batir a Random Forest.

Datos tabulares

Su terreno natural.

Relaciones complejas

No linealidad e interacciones sutiles.

Señal + ruido moderado

Con regularización adecuada.

Feature importance / SHAP

Explicabilidad post-hoc disponible.

Con tiempo de tuning

Recompensa el ajuste cuidadoso.

Gradient Boosting: balance

VENTAJAS

- ✓ Precisión de referencia en tabular
- ✓ Capta interacciones complejas
- ✓ Regularización incorporada
- ✓ Importancia de variables y SHAP

LIMITACIONES

- ✗ Más hiperparámetros que ajustar
- ✗ Entrenamiento secuencial (más lento)
- ✗ Riesgo de sobreajuste sin control
- ✗ Menos interpretable que modelos lineales

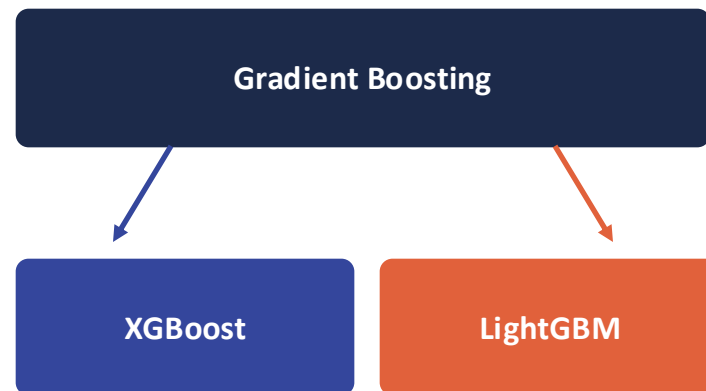
EJEMPLO REAL • PRECIO DE VIVIENDAS

En el benchmark `california_housing`, el boosting baja el RMSE de ~68.900 (lineal) a ~47.300: casi un 31 % menos de error.

XGBoost y LightGBM

Implementaciones avanzadas y optimizadas de Gradient Boosting, hoy estándar en la industria.

- XGBoost: regularización fuerte, muy robusto y probado.
- LightGBM: histogramas y crecimiento por hojas → velocidad.
- Manejan valores faltantes y escalan a millones de filas.



(y CatBoost para variables categóricas)

Por qué dominan las competencias

75%+

de las soluciones ganadoras en datos tabulares (Kaggle, 2015–2024) usan gradient boosting.

Manejo nativo de faltantes

Aprende la mejor dirección para los NaN.

Captura interacciones

No linealidades automáticas entre variables.

Regularización integrada

γ y λ controlan la complejidad.

Robusto y sin escalado

Insensible a la escala y a outliers.

Defaults muy sólidos

Gran rendimiento casi de fábrica.

Interpretabilidad post-hoc

Feature importance y valores SHAP.

Cuándo usar XGBoost / LightGBM

Cuando trabajas con datos tabulares y la precisión es la prioridad —de prototipo a producción.

Datos tabulares

Filas y columnas: su punto fuerte.

Grandes volúmenes

LightGBM entrena rapidísimo a gran escala.

Máxima precisión

Referencia en Kaggle y en la industria.

Muchas features

Numéricas y categóricas mezcladas.

Producción

Rápidos en inferencia, bien soportados.

Faltantes y outliers

Los gestionan de forma nativa.

LightGBM brilla con datos grandes y numéricos; XGBoost, muy robusto y afinable; CatBoost, con categóricas.

XGBoost / LightGBM: balance

VENTAJAS

- ✓ Precisión estado del arte en tabular
- ✓ Muy rápidos y escalables (LightGBM)
- ✓ Faltantes y regularización nativos
- ✓ Enorme ecosistema y soporte

LIMITACIONES

- ✗ Muchos hiperparámetros que ajustar
- ✗ LightGBM puede sobreajustar en datos pequeños
- ✗ Menos interpretable que la lineal
- ✗ Curva de aprendizaje para tuning fino

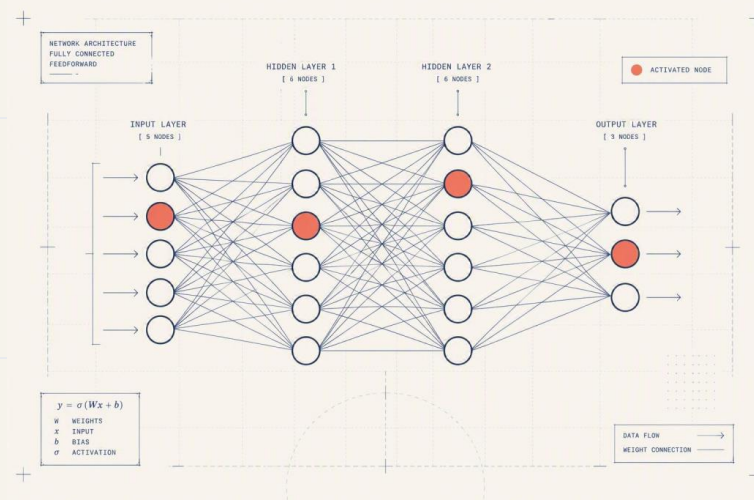
EJEMPLO REAL • PRECIO DE VIVIENDAS

En california_housing, XGBoost (~47.254 RMSE) y LightGBM (~47.488) lideran los modelos clásicos, muy por delante del baseline lineal.

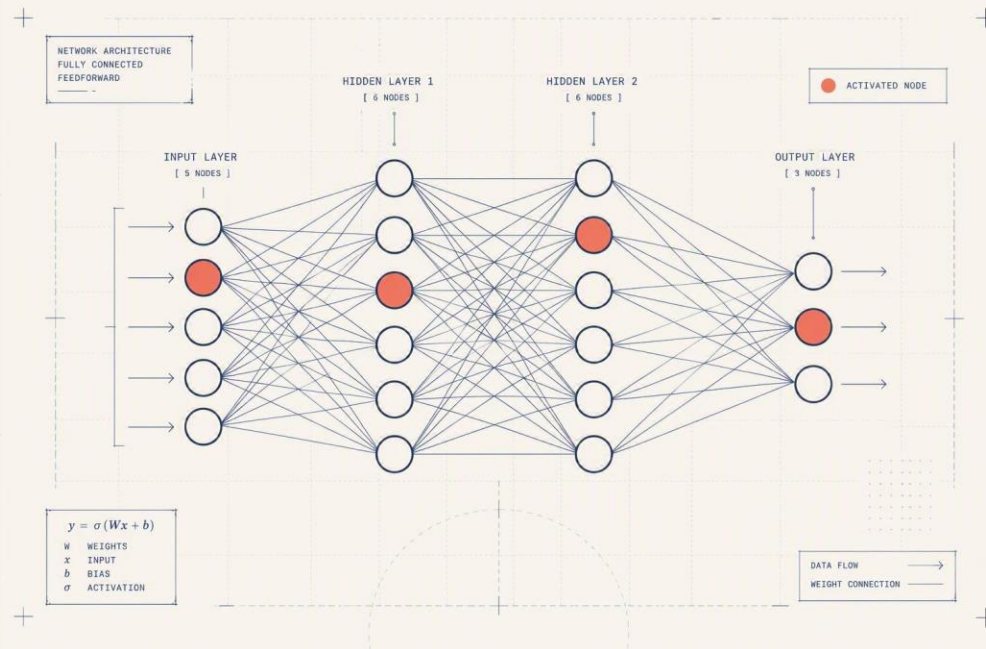
MLPRegressor: perceptrón multicapa

Una red de neuronas que aprende representaciones internas para predecir un valor continuo.

- Capas de neuronas conectadas por pesos ajustables.
- Puede aproximar casi cualquier función.
- Aprende features complejas de forma automática.



Capas, pesos y activaciones



El flujo

- Entrada: características de la casa
- Capas ocultas: combinan y transforman
- Pesos: se ajustan con backpropagation
- Activaciones: introducen no linealidad
- Salida: el precio estimado

Cuándo usar Redes Neuronales

Cuando los patrones son muy complejos y dispones de muchos datos y cómputo.

Patrones complejos

Interacciones muy no lineales.

Muchos datos

Decenas de miles de ejemplos o más.

Datos no estructurados

Texto, imágenes, señales junto a lo tabular.

Feature learning

Aprender representaciones automáticamente.

Con GPU / tiempo

Recursos para entrenar y ajustar.

Modelos multi-salida

Predecir varias magnitudes a la vez.

Redes Neuronales: balance

VENTAJAS

- ✓ Aproximan funciones muy complejas
- ✓ Aprenden features automáticamente
- ✓ Escalan a datos no estructurados
- ✓ Flexibles y multi-salida

LIMITACIONES

- ✗ Necesitan muchos datos y cómputo
- ✗ Difíciles de interpretar (caja negra)
- ✗ Muy sensibles al tuning
- ✗ A menudo excesivas en datos pequeños

EJEMPLO REAL • PRECIO DE VIVIENDAS

En datos tabulares, dos benchmarks grandes (111 y 300+ datasets) muestran que el deep learning suele igualar o quedar por debajo del boosting.

¿Cómo evaluar un modelo de regresión?

¿Por qué no existe “accuracy”?

En regresión predecimos un número continuo: casi nunca acertamos el valor exacto.

No tiene sentido preguntar “¿acertó?”, sino “¿por cuánto se equivocó?”.

Medimos la magnitud del error entre $\hat{y}$ y el valor real y .

MAE

Error absoluto medio

MSE / RMSE

Penaliza errores grandes

R^2

Varianza explicada

MAE — Error Absoluto Medio

$$\text{MAE} = (1/n) \cdot \sum |y_i - \hat{y}_i|$$

- Promedio de los errores en valor absoluto.
- En las mismas unidades del objetivo (€).
- Trata todos los errores por igual.
- Robusto frente a outliers.

42,8

MAE de una regresión lineal sobre el dataset diabetes (sklearn)

EJEMPLO REAL • PRECIO DE VIVIENDAS

Si el MAE es 15.000 €, de media el modelo se equivoca 15.000 € por casa —una lectura directa e intuitiva para negocio.

MSE y RMSE — penalizar el error grande

$$\text{MSE} = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- Elevan el error al cuadrado: castigan los fallos grandes.
- RMSE vuelve a las unidades originales (€).
- Muy sensibles a los outliers.
- Ideales cuando un error grande es muy costoso.

Un error se multiplica al elevarlo

Predigo 300.000 € · real 320.000 €

Error = 20.000 €

$$\text{Error}^2 = 400.000.000$$

Fallar 20 k€ pesa mucho más que dos fallos de 10 k€ → el modelo evita los errores gruesos.

R² — Coeficiente de determinación

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

- Proporción de la varianza del precio que el modelo explica.
- Adimensional: permite comparar modelos.
- R² ajustado penaliza variables inútiles.

1,0

Predicción perfecta

0,7-0,9

Buen ajuste

0,0

Igual que predecir la media

< 0

Peor que la media

Tabla comparativa completa

Modelo	Interpret.	Complej.	Overfit	Escalab.	No lineal	Veloc.	Caso recomendado
Lineal	Muy alta	Baja	Bajo	Alta	No	Muy alta	Base interpretable
Polinómica	Alta	Media	Alto	Media	Sí (suave)	Alta	Curvas simples
Ridge	Alta	Baja	Bajo	Alta	No	Muy alta	Colinealidad
Lasso	Muy alta	Baja	Bajo	Alta	No	Muy alta	Selección variables
Elastic Net	Alta	Media	Bajo	Alta	No	Alta	Muchas vars. corr.
Árbol	Alta	Media	Muy alto	Media	Sí	Alta	Reglas visuales
Random Forest	Media	Alta	Bajo	Alta	Sí	Media	Base robusta
SVR	Baja	Alta	Medio	Baja	Sí (RBF)	Baja	Datos peq/medianos
KNN	Media	Baja	Medio	Baja	Sí	Baja	Patrones locales
Gradient Boost.	Media	Alta	Medio	Alta	Sí	Media	Alta precisión
XGBoost	Media	Alta	Medio	Muy alta	Sí	Alta	Tabular competición
LightGBM	Media	Alta	Medio	Muy alta	Sí	Muy alta	Gran volumen
MLP (red)	Baja	Muy alta	Alto	Alta	Sí	Baja	Patrones complejos

¿Qué modelo elegir?

Si necesito explicar

Lineal

Ridge

Lasso

Relaciones no lineales

Árbol

Random Forest

Gradient Boosting

Máxima precisión

XGBoost

LightGBM

Pocos datos

Lineal

Ridge

SVR

Muchas variables

Lasso

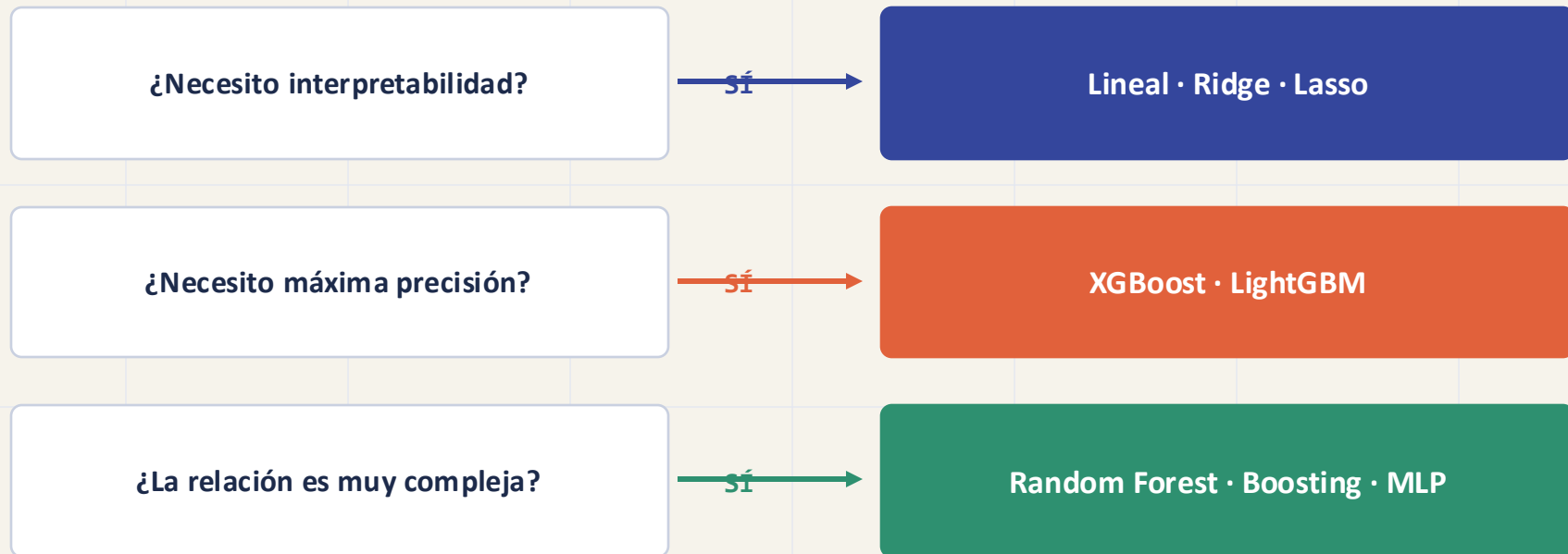
Elastic Net

Gran volumen de datos

LightGBM

XGBoost

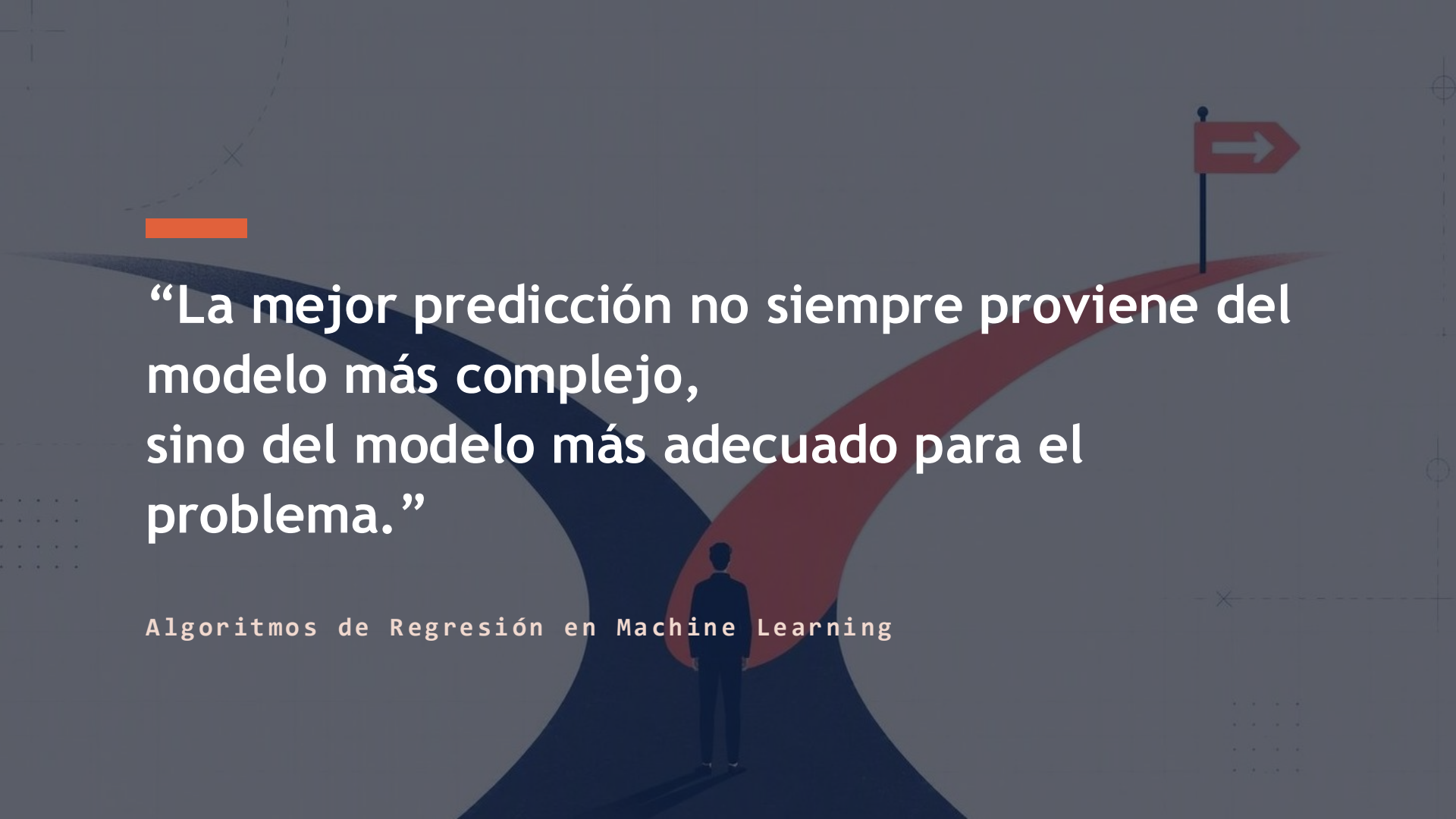
Árbol para elegir un algoritmo



Recorre las preguntas de arriba abajo: la primera que responda “sí” marca tu punto de partida. Empieza simple y sube en complejidad sólo si el problema lo exige.

Conclusiones

- 1 No hay algoritmo universal** El “mejor” depende del problema, no existe uno que gane siempre.
- 2 La elección depende de los datos** Volumen, dimensionalidad, ruido y tipo de relación mandan.
- 3 Interpretabilidad = precisión** Explicar el porqué suele importar tanto como acertar.
- 4 Lo complejo no siempre gana** En tabular, el boosting bate a menudo a las redes neuronales.
- 5 Empieza simple** Una buena línea base guía y valida antes de sofisticar.

The background is a dark gray gradient. A large, stylized, curved path in shades of blue and red leads from the bottom center towards the top right. A small silhouette of a person stands on the path. At the end of the path, there is a red signpost with a white arrow pointing right. In the top left corner, there is a small orange rectangle and some faint, light gray geometric lines and dots.

“La mejor predicción no siempre proviene del modelo más complejo, sino del modelo más adecuado para el problema.”

Algoritmos de Regresión en Machine Learning